

# SDD Competitive Analysis

Ed Barlow    [www.webcloudstudio.com](http://www.webcloudstudio.com)    August 5, 2026



## SDD Competitive Analysis

| Key | Product         | What it is                                                       | License     | Runs on               |
|-----|-----------------|------------------------------------------------------------------|-------------|-----------------------|
| DD  | Drydock         | Specification builder based on Agile and Test Driven Development | MIT         | Your LLM subscription |
| SK  | GitHub Spec Kit | Prompt templates turning an idea into spec, plan, and tasks      | MIT         | Your LLM subscription |
| KI  | Kiro (AWS)      | IDE with a spec-first mode and agent hooks                       | Proprietary | Metered inference     |
| BM  | BMAD-METHOD     | Multi-agent role framework for elicitation and story delivery    | MIT         | Your LLM subscription |
| CM  | Plain CLAUDE.md | One rules file and an agent                                      | Free        | Your LLM subscription |

## Summary

| Section                  | DD                         | SK                         | KI                         | BM                         | CM                         | Winner    |
|--------------------------|----------------------------|----------------------------|----------------------------|----------------------------|----------------------------|-----------|
| 1 Specification handling | <div><div></div></div> 3.8 | <div><div></div></div> 2.0 | <div><div></div></div> 2.6 | <div><div></div></div> 3.0 | <div><div></div></div> 1.2 | Drydock   |
| 2 Planning               | <div><div></div></div> 5.0 | <div><div></div></div> 1.4 | <div><div></div></div> 2.0 | <div><div></div></div> 2.6 | <div><div></div></div> 0.2 | Drydock   |
| 3 Build execution        | <div><div></div></div> 4.2 | <div><div></div></div> 1.0 | <div><div></div></div> 1.7 | <div><div></div></div> 2.5 | <div><div></div></div> 1.2 | Drydock   |
| 4 Verification           | <div><div></div></div> 4.8 | <div><div></div></div> 0.3 | <div><div></div></div> 1.2 | <div><div></div></div> 1.2 | <div><div></div></div> 0.7 | Drydock   |
| 5 Change management      | <div><div></div></div> 4.8 | <div><div></div></div> 0.6 | <div><div></div></div> 1.2 | <div><div></div></div> 1.2 | <div><div></div></div> 0.2 | Drydock   |
| 6 Context and cost       | <div><div></div></div> 4.8 | <div><div></div></div> 1.3 | <div><div></div></div> 0.5 | <div><div></div></div> 2.3 | <div><div></div></div> 1.8 | Drydock   |
| 7 Governance             | <div><div></div></div> 4.8 | <div><div></div></div> 1.5 | <div><div></div></div> 2.0 | <div><div></div></div> 1.8 | <div><div></div></div> 2.3 | Drydock   |
| 8 Review surface         | <div><div></div></div> 4.8 | <div><div></div></div> 0.5 | <div><div></div></div> 2.5 | <div><div></div></div> 1.8 | <div><div></div></div> 0.3 | Drydock   |
| 9 Adoption cost          | <div><div></div></div> 1.8 | <div><div></div></div> 3.7 | <div><div></div></div> 2.7 | <div><div></div></div> 3.3 | <div><div></div></div> 5.0 | CLAUDE.md |
| <b>Overall</b>           | <div><div></div></div> 4.2 | <div><div></div></div> 1.4 | <div><div></div></div> 1.8 | <div><div></div></div> 2.2 | <div><div></div></div> 1.5 | Drydock   |

## 1 Specification Handling

| #   | Capability                           | DD  | SK  | KI  | BM  | CM  | Winner  |
|-----|--------------------------------------|-----|-----|-----|-----|-----|---------|
| 1.1 | Authors a specification from scratch | ● 0 | ● 4 | ● 4 | ● 5 | ● 2 | BMAD    |
| 1.2 | Ingests existing source material     | ● 5 | ● 1 | ● 2 | ● 3 | ● 2 | Drydock |
| 1.3 | Typed specification files            | ● 5 | ● 3 | ● 3 | ● 3 | ● 1 | Drydock |
| 1.4 | Specification quality audit          | ● 4 | ● 0 | ● 1 | ● 2 | ● 0 | Drydock |
| 1.5 | Specification stays authoritative    | ● 5 | ● 2 | ● 3 | ● 2 | ● 1 | Drydock |

BMAD is the clear winner because its analyst and product-manager agents produce a specification from nothing. Drydock builds from specifications but does not author specifications.

## 2 Planning

| #   | Capability                               | DD  | SK  | KI  | BM  | CM  | Winner  |
|-----|------------------------------------------|-----|-----|-----|-----|-----|---------|
| 2.1 | Dependency graph database                | ● 5 | ● 1 | ● 1 | ● 2 | ● 0 | Drydock |
| 2.2 | Stories with stable IDs enabling rebuild | ● 5 | ● 3 | ● 4 | ● 4 | ● 0 | Drydock |
| 2.3 | Effort estimated as token cost           | ● 5 | ● 0 | ● 0 | ● 2 | ● 0 | Drydock |
| 2.4 | Blocker gate halts planning              | ● 5 | ● 1 | ● 2 | ● 3 | ● 0 | Drydock |
| 2.5 | Human-editable plan                      | ● 5 | ● 2 | ● 3 | ● 2 | ● 1 | Drydock |

Drydock is the clear winner because its plan is a queryable dependency graph, not an ordered task list.

## 3 Build Execution

| #   | Capability                            | DD  | SK  | KI  | BM  | CM  | Winner  |
|-----|---------------------------------------|-----|-----|-----|-----|-----|---------|
| 3.1 | Executes the plan to working software | ● 5 | ● 3 | ● 4 | ● 4 | ● 3 | Drydock |
| 3.2 | Per-step context stacking             | ● 5 | ● 2 | ● 2 | ● 4 | ● 1 | Drydock |
| 3.3 | Automatic repair loop                 | ● 5 | ● 0 | ● 1 | ● 2 | ● 1 | Drydock |
| 3.4 | Model escalation on final retry       | ● 5 | ● 0 | ● 0 | ● 1 | ● 0 | Drydock |
| 3.5 | Parallel execution                    | ● 0 | ● 0 | ● 1 | ● 2 | ● 1 | BMAD    |
| 3.6 | Resume in place after failure         | ● 5 | ● 1 | ● 2 | ● 2 | ● 1 | Drydock |

Drydock is the clear winner on repair and resumption. BMAD wins parallel execution; Drydock scores zero on it.

## 4 Verification

| #   | Capability                     | DD  | SK  | KI  | BM  | CM  | Winner  |
|-----|--------------------------------|-----|-----|-----|-----|-----|---------|
| 4.1 | TDD gate before implementation | ● 4 | ● 1 | ● 2 | ● 3 | ● 2 | Drydock |
| 4.2 | Acceptance executed, no LLM    | ● 5 | ● 0 | ● 1 | ● 1 | ● 0 | Drydock |

| #   | Capability                    | DD  | SK  | KI  | BM  | CM  | Winner  |
|-----|-------------------------------|-----|-----|-----|-----|-----|---------|
| 4.3 | Vacuous proofs demoted        | ● 5 | ● 0 | ● 0 | ● 0 | ● 0 | Drydock |
| 4.4 | EARS criteria with stable IDs | ● 5 | ● 0 | ● 3 | ● 0 | ● 0 | Drydock |
| 4.5 | Guardrails behind a hard gate | ● 5 | ● 1 | ● 0 | ● 1 | ● 2 | Drydock |
| 4.6 | Release verdict with blockers | ● 5 | ● 0 | ● 1 | ● 2 | ● 0 | Drydock |

Drydock is the clear winner because acceptance criteria are executed rather than judged. It scores 4 on 4.1 because proofs are not hash-locked before implementation.

## 5 Change Management

| #   | Capability                        | DD  | SK  | KI  | BM  | CM  | Winner  |
|-----|-----------------------------------|-----|-----|-----|-----|-----|---------|
| 5.1 | Drift detection by content hash   | ● 5 | ● 0 | ● 1 | ● 0 | ● 0 | Drydock |
| 5.2 | Sealed specs need a change ticket | ● 5 | ● 0 | ● 0 | ● 1 | ● 0 | Drydock |
| 5.3 | Replan scoped to impacted work    | ● 5 | ● 1 | ● 2 | ● 2 | ● 0 | Drydock |
| 5.4 | Change spec, then rebuild         | ● 5 | ● 2 | ● 3 | ● 3 | ● 1 | Drydock |
| 5.5 | Regression reopens sibling story  | ● 4 | ● 0 | ● 0 | ● 0 | ● 0 | Drydock |

Drydock is the clear winner because it is the only product here that detects specification drift and replans against it.

## 6 Context and Cost

| #   | Capability                       | DD  | SK  | KI  | BM  | CM  | Winner              |
|-----|----------------------------------|-----|-----|-----|-----|-----|---------------------|
| 6.1 | Specification compaction         | ● 5 | ● 0 | ● 0 | ● 3 | ● 1 | Drydock             |
| 6.2 | Compact selection by build graph | ● 5 | ● 0 | ● 0 | ● 0 | ● 0 | Drydock             |
| 6.3 | No API-key path                  | ● 5 | ● 3 | ● 0 | ● 3 | ● 5 | Drydock / CLAUDE.md |
| 6.4 | Runs on a cheaper model          | ● 4 | ● 2 | ● 2 | ● 3 | ● 1 | Drydock             |

Drydock is the clear winner because context is compacted and priced per story. Kiro is last: it is the only product billed per token.

## 7 Governance

| #   | Capability                  | DD  | SK  | KI  | BM  | CM  | Winner  |
|-----|-----------------------------|-----|-----|-----|-----|-----|---------|
| 7.1 | Injected rules and branding | ● 5 | ● 2 | ● 3 | ● 2 | ● 4 | Drydock |
| 7.2 | Persistent intent injection | ● 5 | ● 3 | ● 3 | ● 3 | ● 4 | Drydock |
| 7.3 | Build evidence per step     | ● 5 | ● 1 | ● 2 | ● 2 | ● 0 | Drydock |
| 7.4 | Dependency legitimacy guard | ● 4 | ● 0 | ● 0 | ● 0 | ● 1 | Drydock |

Drydock is the clear winner on injected governance and evidence. CLAUDE.md places second on rules injection with no tooling at all.

## 8 Review Surface

| #   | Capability                | DD  | SK  | KI  | BM  | CM  | Winner  |
|-----|---------------------------|-----|-----|-----|-----|-----|---------|
| 8.1 | Review console            | ● 5 | ● 0 | ● 4 | ● 1 | ● 0 | Drydock |
| 8.2 | Persistent questionnaires | ● 5 | ● 1 | ● 2 | ● 3 | ● 0 | Drydock |
| 8.3 | Board view of plan state  | ● 4 | ● 0 | ● 3 | ● 1 | ● 0 | Drydock |
| 8.4 | Documentation generation  | ● 5 | ● 1 | ● 1 | ● 2 | ● 1 | Drydock |

Drydock is the clear winner because its console reviews build evidence. Kiro's console reviews the specification only.

## 9 Adoption Cost

| #   | Capability                     | DD  | SK  | KI  | BM  | CM  | Winner               |
|-----|--------------------------------|-----|-----|-----|-----|-----|----------------------|
| 9.1 | Time to first working code     | ● 1 | ● 3 | ● 4 | ● 2 | ● 5 | CLAUDE.md            |
| 9.2 | Right-sized for a one-file fix | ● 1 | ● 2 | ● 3 | ● 3 | ● 5 | CLAUDE.md            |
| 9.3 | Zero install                   | ● 1 | ● 3 | ● 1 | ● 2 | ● 5 | CLAUDE.md            |
| 9.4 | Open source                    | ● 5 | ● 5 | ● 1 | ● 5 | ● 5 | All but Kiro         |
| 9.5 | Stack coverage out of the box  | ● 2 | ● 4 | ● 4 | ● 4 | ● 5 | CLAUDE.md            |
| 9.6 | Community and ecosystem        | ● 1 | ● 5 | ● 3 | ● 4 | ● 5 | Spec Kit / CLAUDE.md |

CLAUDE.md is the clear winner because it costs nothing to adopt. Drydock is last: Python rigging only, no ecosystem, and a four-command path to first running software.

## Which To Use

| Situation                                          | Use             |
|----------------------------------------------------|-----------------|
| One-file fix or exploratory hacking                | Plain CLAUDE.md |
| Nothing written down yet                           | BMAD-METHOD     |
| Free, agent-neutral team convention                | GitHub Spec Kit |
| IDE-centric team, spec-first with editor hooks     | Kiro            |
| Large system, spec already written, months of work | Drydock         |
| Delivery that must be audited or defended          | Drydock         |
| Long-lived product whose spec keeps changing       | Drydock         |
| Maximum agent throughput on independent work       | None of these   |

The natural pairing is BMAD or Kiro upstream, Drydock downstream.

## The Wider Field

Eleven systems on the same scale. Maturity follows Böckeler's taxonomy: **spec-first** (the spec guides the first build, then drifts), **spec-anchored** (spec and code co-evolve under tests), **spec-as-source** (humans edit only specs).

| Key | Product                      | License     | Maturity       |
|-----|------------------------------|-------------|----------------|
| DD  | Drydock                      | MIT         | Spec-anchored  |
| SK  | GitHub Spec Kit              | MIT         | Spec-first     |
| KI  | Kiro (AWS)                   | Proprietary | Spec-first     |
| TS  | Tessl + intent-integrity-kit | Proprietary | Spec-as-source |
| OS  | OpenSpec                     | MIT         | Spec-anchored  |
| BM  | BMAD-METHOD                  | MIT         | Spec-first     |
| KT  | Spec Kitty                   | Open source | Spec-anchored  |
| WA  | Walden                       | Open source | Spec-anchored  |
| SP  | Superpowers                  | Open source | Spec-first     |
| GS  | GSD (Get Shit Done)          | Open source | Spec-anchored  |
| TR  | Traycer                      | Proprietary | Spec-first     |

| #  | Feature                              | DD | SK | KI | TS | OS | BM | KT | WA | SP | GS | TR |
|----|--------------------------------------|----|----|----|----|----|----|----|----|----|----|----|
| 1  | Typed specification files            | 5  | 3  | 3  | 4  | 3  | 3  | 3  | 3  | 1  | 3  | 2  |
| 2  | Dependency graph database            | 5  | 1  | 1  | 1  | 1  | 2  | 2  | 2  | 2  | 4  | 2  |
| 3  | Context-aware prompt stacking        | 5  | 1  | 1  | 2  | 2  | 4  | 2  | 1  | 3  | 4  | 3  |
| 4  | Story points as token cost           | 5  | 0  | 0  | 0  | 0  | 2  | 0  | 0  | 0  | 1  | 0  |
| 5  | EARS criteria, grammar-validated     | 5  | 0  | 2  | 0  | 0  | 0  | 0  | 5  | 0  | 0  | 0  |
| 6  | Guardrails behind a hard gate        | 5  | 2  | 0  | 2  | 0  | 0  | 1  | 3  | 0  | 2  | 0  |
| 7  | Evidence bound to Git and hashes     | 5  | 0  | 0  | 3  | 0  | 0  | 1  | 4  | 1  | 2  | 0  |
| 8  | Model-judged verification discounted | 5  | 0  | 0  | 2  | 0  | 0  | 0  | 2  | 0  | 0  | 0  |
| 9  | Drift detection by spec hashing      | 4  | 0  | 0  | 4  | 2  | 0  | 1  | 5  | 0  | 2  | 1  |
| 10 | Sealed specs need a ticket           | 5  | 1  | 0  | 2  | 3  | 0  | 1  | 3  | 0  | 1  | 0  |
| 11 | Review console                       | 4  | 1  | 4  | 2  | 3  | 2  | 4  | 2  | 1  | 1  | 5  |
| 12 | Persistent intent injection          | 5  | 4  | 3  | 3  | 3  | 3  | 3  | 4  | 2  | 3  | 2  |
| 13 | Branding and stack rules injection   | 5  | 2  | 3  | 1  | 1  | 2  | 1  | 2  | 1  | 1  | 1  |
| 14 | Specification compaction             | 5  | 0  | 0  | 2  | 1  | 3  | 0  | 0  | 1  | 2  | 1  |
| 15 | Blocker gate halts planning          | 5  | 1  | 2  | 1  | 2  | 3  | 2  | 3  | 3  | 3  | 2  |
| 16 | Brownfield reverse-engineering       | 3  | 1  | 2  | 2  | 4  | 3  | 2  | 1  | 1  | 2  | 3  |
| 17 | Documentation generation             | 5  | 1  | 1  | 2  | 1  | 2  | 1  | 1  | 0  | 1  | 1  |
| 18 | Subscription-CLI-only execution      | 5  | 3  | 0  | 0  | 3  | 3  | 3  | 4  | 4  | 3  | 0  |
| 19 | Provider and IDE neutrality          | 4  | 5  | 0  | 2  | 4  | 4  | 5  | 5  | 2  | 2  | 3  |
| 20 | Worktree isolation for agents        | 0  | 0  | 0  | 0  | 1  | 2  | 5  | 0  | 5  | 4  | 3  |

| #  | Feature                            | DD | SK | KI | TS | OS | BM | KT | WA | SP | GS | TR |
|----|------------------------------------|----|----|----|----|----|----|----|----|----|----|----|
| 21 | Parallel execution                 | 0  | 0  | 0  | 0  | 1  | 3  | 5  | 0  | 4  | 5  | 3  |
| 22 | Enforced TDD RED→GREEN cycle       | 4  | 1  | 2  | 3  | 1  | 3  | 2  | 3  | 5  | 3  | 1  |
| 23 | Tests hash-locked before build     | 0  | 0  | 0  | 5  | 0  | 0  | 0  | 2  | 1  | 0  | 0  |
| 24 | Test-quality analysis              | 5  | 0  | 0  | 5  | 0  | 0  | 0  | 1  | 2  | 2  | 0  |
| 25 | Delta change format with archive   | 2  | 1  | 1  | 2  | 5  | 1  | 2  | 2  | 0  | 2  | 1  |
| 26 | CI release gate with exit code     | 1  | 1  | 0  | 2  | 2  | 0  | 2  | 5  | 0  | 2  | 0  |
| 27 | Semantic diff and impact warning   | 1  | 0  | 0  | 5  | 3  | 0  | 1  | 4  | 0  | 2  | 1  |
| 28 | Retrospectives and lessons capture | 1  | 0  | 0  | 1  | 1  | 2  | 5  | 4  | 2  | 2  | 1  |
| 29 | Workflow right-sized to problem    | 1  | 1  | 1  | 2  | 4  | 5  | 2  | 1  | 3  | 4  | 4  |
| 30 | Spike phase before planning        | 2  | 2  | 1  | 3  | 5  | 4  | 2  | 2  | 5  | 5  | 4  |
| 31 | Cross-feature conflict analysis    | 1  | 1  | 1  | 2  | 2  | 3  | 1  | 2  | 1  | 2  | 2  |
| 32 | Decision-coverage gate             | 1  | 0  | 0  | 3  | 1  | 1  | 1  | 2  | 1  | 5  | 1  |
| 33 | Malicious package detection        | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 5  | 0  |
| 34 | Model routing and cost tiering     | 1  | 0  | 0  | 1  | 0  | 2  | 1  | 0  | 2  | 5  | 3  |
| 35 | Gherkin behavioral specs           | 0  | 1  | 2  | 4  | 3  | 2  | 2  | 2  | 1  | 1  | 1  |
| 36 | Spec bloat control                 | 2  | 1  | 1  | 2  | 3  | 1  | 2  | 3  | 2  | 2  | 2  |
| 37 | Cross-repository spec sharing      | 0  | 0  | 0  | 4  | 3  | 1  | 0  | 0  | 0  | 0  | 0  |
| 38 | Fix spec first, regenerate code    | 2  | 1  | 1  | 5  | 3  | 1  | 2  | 3  | 1  | 2  | 1  |
| 39 | Multi-agent role specialization    | 1  | 2  | 2  | 2  | 1  | 5  | 4  | 1  | 4  | 4  | 3  |

Walden is the closest competitor to Drydock, reaching EARS validation, staleness chains, and proof-based completion independently, and winning the CI release gate outright.

Tessl wins test integrity by hash-locking tests before implementation. Spec Kitty, Superpowers, and GSD win parallel execution. BMAD wins right-sizing.

Rows 22, 24, and 33 changed since the 2026-07-17 edition, when Drydock scored 1, 0, and 0.

## Drydock Gaps

1. **Right-sizing.** No lightweight path. A one-line change pays the full pipeline.
2. **Parallel execution.** Scores zero while holding the dependency graph that would make it safe.
3. **Authoring.** A stated non-goal, so Drydock cannot be a team's only tool.
4. **Stack coverage.** Python rigging only.
5. **Proof locking.** Criteria are gated but not hash-locked before implementation.
6. **CI gate.** The release verdict is not packaged as an exit code a pipeline can enforce.

## Legend

---

| Mark | Score | Meaning                                   |
|------|-------|-------------------------------------------|
| ●    | 5     | Best in field; deterministic and enforced |
| ●    | 4     | Strong first-class capability             |
| ●    | 3     | Present; advisory rather than enforced    |
| ●    | 2     | Partial or manual                         |
| ●    | 1     | Nominal or documentation-only             |
| ●    | 0     | Absent                                    |

## Method

---

Analysis date 2026-08-05, superseding the 2026-07-17 edition. Drydock is scored against its published specification. Every other product is scored from published primary documentation; none was executed. Scores are engineering judgment, not measurements.

Sections 1–8 are drawn from the problem Drydock solves, which biases the summary in its favor. Section 9 and the gap list are the counterweight.

## Sources

---

This is an honest analysis by Claude of the following documents to compare these products.

- Web Cloud Studio — <https://www.webcloudstudio.com>
- Drydock Specification — [https://webcloudstudio.github.io/Drydock/Drydock\\_Specification.pdf](https://webcloudstudio.github.io/Drydock/Drydock_Specification.pdf)
- GitHub Spec Kit — <https://github.com/github/spec-kit>
- Kiro — <https://kiro.dev/docs/>
- BMAD-METHOD — <https://docs.bmad-method.org/>
- OpenSpec — <https://github.com/Fission-AI/OpenSpec>
- Spec Kitty — <https://github.com/Priivacy-ai/spec-kitty>
- Walden — <https://andrearaponi.github.io/walden/>
- Superpowers — <https://github.com/obra/superpowers>
- GSD — <https://github.com/gsd-build/get-shit-done>
- Traycer — <https://docs.traycer.ai/quickstart>
- Tessl intent-integrity-kit — <https://tessl.io/registry/tessl-labs/intent-integrity-kit/2.7.5>
- Böckeler, *Understanding Spec-Driven Development: Kiro, spec-kit, and Tessl* — <https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html>
- *Spec-Driven Development: From Code to Contract* — <https://arxiv.org/html/2602.00180v1>
- cameronjsj/spec-compare — <https://github.com/cameronjsj/spec-compare>

- *Skepticism Toward Specification-Driven Development* —  
<https://zenn.dev/cbmrham/articles/202601-spec-driven-development-skepticism>